

Wireshark, Atrasos, HTTP (Linux ou Windows)

OBS: NÃO utilize modo anônimo do navegador nesse laboratório, pois senão o navegador transforma em HTTPS, e não é o objetivo hoje.

OBS2: Efetue preventivamente uma limpeza de cache no navegador. Normalmente nas configurações de segurança (três pontinhos verticais + privacidade e segurança + excluir dados).

1. No wireshark, faça o seguinte:

- Entendimento das 3 janelas do wireshark (pacotes, detalhes de um pacote e bytes)
- Criar seu próprio perfil. Embaixo à direita, clique da direita em “Perfil” + “Novo”. Crie um novo perfil que comece pelo seu nome ou algum identificador de preferência. Trabalhe nesse perfil durante as aulas. Você pode ir em “Gerenciar Perfis” e exportar especificamente seu perfil pessoal para uso em aula futura.
- Mudar o layout das janelas conforme o gosto (Editar + Preferências + Aparência + Layout).
- Mudar a cor de fundo dos pacotes enviados pela máquina origem. DICA: “exibir + colorindo regras”. Criar uma regra para `ip.src==...`
- Mudar a cor de fundo dos pacotes SYN. DICA: Regra de cores para `tcp.flags.syn==1`
- Incluir uma coluna com o tempo “Delta” (tempo desde o último pacote). DICA: “Editar + Preferências + Colunas” ou clique da direita no início do pacote onde diz *[Time delta from previous displayed frame]* e *add as column*.

2. Capture via wireshark uma consulta HTTP para <http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html>.

- Listar os diferentes protocolos de um pacote HTTP no laboratório, relacionando os mesmos com o modelo de camadas da Internet.
- Detalhe a formação do pacote, identificando cada protocolo do modelo Internet.
- Qual o RTT (*Round Trip Time*) aproximado da conexão?
- Descobrir o atraso entre o **HTTP GET** e o correspondente **200 OK** (no wireshark identificar o pacote correto).
- Qual o tamanho do arquivo baixado para a máquina?

3. Capture via wireshark uma consulta HTTP para <http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file3.html>. Responda, para o HTTP:

OBS IMPORTANTE: para filtrar um fluxo TCP de interesse, uma forma é localizar um pacote desse fluxo, fazer clique da direita e “seguir + fluxo TCP”. Outro filtro interessante é simplesmente “http”.

- Qual o código de status HTTP de retorno para o GET do arquivo?
- Qual a última data de modificação do arquivo?
- Qual o tamanho do cabeçalho HTTP no pacote de resposta?
- Qual o tamanho do arquivo baixado para a máquina?
- Baixe novamente o mesmo arquivo sem apagar a cache do navegador (**utilizando F5**). Verifique agora se a mensagem continua sendo “200 OK” ou mudou. Explique. OBS: se não aparecer outro HTTP GET na captura, pode ser porque o navegador utilizou o cache sem perguntar ao servidor (ele pode assumir que a validade do cache é ~10% do tempo em relação à data “Last Modified”. Isso normalmente é resolvido atualizando com F5 em vez de abrir uma nova aba e utilizar o endereço todo novamente.

4. Execute a animação referente à estimativa de atraso no HTTP. Acesso em https://media.pearsoncmg.com/aw/ecs_kurose_compnetwork_7/cw/content/interactiveanimations/http-delay-estimation/index.html

- Escolher “*non-persistent connections*” (HTTP 1.0) com 4 objetos e atraso de transmissão para cada objeto de 1 RTT. Comentar. O HTTP 1.0 permitia também conexões em paralelo, abrindo múltiplas conexões (ex: 6), onde cada conexão baixava um arquivo.
- Escolher “*persistent connections with pipeline*”. O restante mantém. Comentar. Essa solução é utilizada no HTTP 1.1 (uma conexão TCP que mantém uma fila ordenada de requisições, gerando o problema do *Head of Line Blocking*. Já o HTTP/2 utiliza multiplexação, ou seja, uma única conexão TCP e vários GETs de objetos dentro da mesma conexão (parecido com pipeline mas não igual, pois não gera *Head of Line Blocking*).
- Debate sobre número de RTTs necessários para cada solução, bem como a possibilidade ou não de bloqueios no caso de perda de pacotes.